Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Institute for Dynamic Systems and Control
IDSC
Institut für Dynamische Systeme und Regelungstechnik

151-0566-00 **Recursive Estimation** (Spring 2026)

Programming Exercise Topic: Recursive Filtering

Issued: April 22, 2026 Due: June 24, 2026

Recursive Estimation for Whale Tracking

Sensor fusion is a typical application of recursive estimation algorithms. Typically, modern tracking systems will utilize a variety of sensors that track different state components, allowing for the overlaying of their information. The adoption of several sensors is motivated by concerns for robustness as well as accuracy. When a sensor has a flaw or receives inaccurate data, other sensors can still provide some information.

You have been brought on to a scientific mission designed to learn more about the hunting patterns of *Cuvier's beaked whale (Ziphius cavirostris)*, the world's deepest diving mammal, which can dive to depths of 3000 meters. As the team's leading expert on recursive estimation, you are tasked with designing two of the algorithms, an Extended Kalman Filter (EKF) and another algorithm of your choice that we have covered in class, to track a Cuvier's beaked whale's relative position and orientation with respect to your scientific ship, which is designated as the coordinate system origin.

Your ship is equipped with an acoustic telemetry system along with a sonar system. The performance of the acoustic telemetry system degrades and eventually fails as a function of the water depth, while the sonar system provides noisier measurements, but up to much deeper depths.

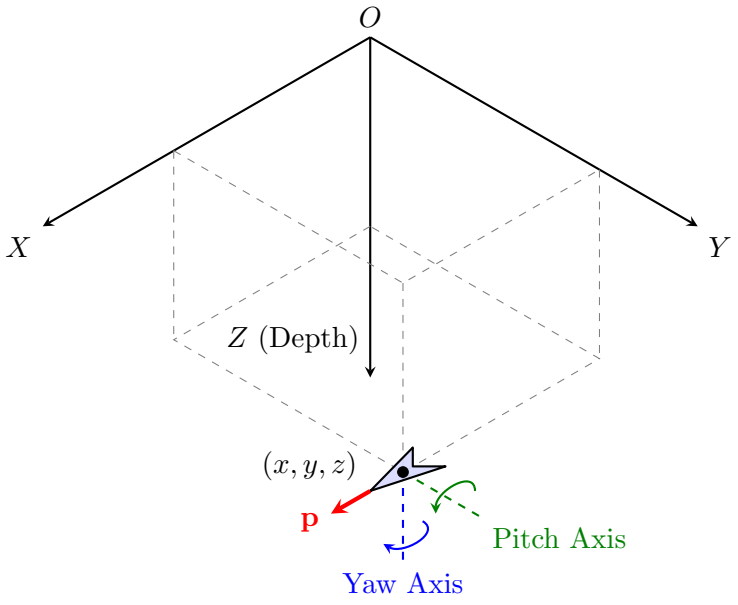

Figure 1: Schematic diagram of the whale you are tracking and the fixed coordinate system (X, Y, Z) with origin O at the scientific ship. The absolute position of the whale is denoted by (x, y, z). The local body frame illustrates the forward velocity (p), the pitch angle (θ), and the yaw angle (ψ).

The whale is an uncooperative target and its precise, high-frequency maneuvering decisions are unknown, however, your scientific team has provided you with a baseline behavioral model of its

diving and cruising habits. Its motion is described by a 9 state discrete time model with time step T_s, where the highest order rates are driven by a known deterministic steering model with additive zero-mean Gaussian process noise:

$$x(k) = x(k\text{–}1) + p(k\text{–}1)\cos(\theta(k\text{–}1))\cos(\psi(k\text{–}1))T_s \tag{1}$$
$$y(k) = y(k\text{–}1) + p(k\text{–}1)\cos(\theta(k\text{–}1))\sin(\psi(k\text{–}1))T_s \tag{2}$$
$$z(k) = \max(0, z(k\text{–}1) + p(k\text{–}1)\sin(\theta(k\text{–}1))T_s)^1 \tag{3}$$
$$\psi(k) = \psi(k\text{–}1) + r_\psi(k\text{–}1)T_s \tag{4}$$
$$\theta(k) = \theta(k\text{–}1) + r_\theta(k\text{–}1)T_s \tag{5}$$
$$p(k) = p(k\text{–}1) + (a(k\text{–}1) - c_d p(k\text{–}1))\,T_s \tag{6}$$
$$r_\psi(k) = r_\psi(k\text{–}1) + K_{\text{yaw}}(\omega_{\text{yaw}} - r_\psi(k\text{–}1)) + v_{r_\psi}(k\text{–}1) \tag{7}$$
$$r_\theta(k) = r_\theta(k\text{–}1) + K_{\theta,p}(\theta^*(k\text{–}1) - \theta(k\text{–}1)) - K_{\theta,d} r_\theta(k\text{–}1) + v_{r_\theta}(k\text{–}1) \tag{8}$$
$$a(k) = a(k\text{–}1) + K_{p,p}(p_{\text{cruise}} - p(k\text{–}1)) - K_{p,d} a(k\text{–}1) + v_a(k\text{–}1) \tag{9}$$

where x and y are the world frame coordinates of the whale, $z \geq 0$ is the whale's depth, ψ is the yaw angle, θ is the pitch angle, and p is the forward velocity. The variables r_ψ, r_θ, and a represent the whale's yaw rate, pitch rate, and forward acceleration, respectively.

The parameters K_{yaw}, $K_{\theta,p}$, $K_{\theta,d}$, $K_{p,p}$, and $K_{p,d}$ are the known behavioral control gains, ω_{yaw} is the target yaw rate, and p_{cruise} is the target cruising speed. A linear drag coefficient c_d is applied to the velocity equation to simulate water resistance. The target pitch angle θ^* represents the whale's dive schedule, switching from a descent to an ascent at a specified fraction of the total time N:

$$\theta^*(k) = \begin{cases} \theta_{\text{dive}} & \text{if } k < N \cdot k_{\text{switch\_frac}}, \\ -\theta_{\text{dive}} & \text{otherwise.} \end{cases} \tag{10}$$

The terms $v_{r_\psi}(k)$, $v_{r_\theta}(k)$, and $v_a(k)$ are zero-mean Gaussian process noises that model the unknowable maneuvering decisions of the whale.

The research vessel receives localization information through a Ultra-Short Baseline (USBL) Acoustic Telemetry system. The measurement model for the USBL sensor is given by:

$$z_{U,x}(k) = x(k) + w_{U,x}(k) \tag{11}$$
$$z_{U,y}(k) = y(k) + w_{U,y}(k) \tag{12}$$

Similarly, the ship's active sonar pings the whale to measure its absolute 3D position. The measurement model for the sonar is given by:

$$z_{S,x}(k) = x(k) + w_{S,x}(k) \tag{13}$$
$$z_{S,y}(k) = y(k) + w_{S,y}(k) \tag{14}$$
$$z_{S,z}(k) = z(k) + w_{S,z}(k) \tag{15}$$

At time $k = 0$, the whale is located at a known starting depth $z(0)$. Its initial horizontal position is modeled as a random vector $[x(0), y(0)]^T$ uniformly distributed over a disk of radius R centered at the origin. The initial physiological states $\psi(0)$, $\theta(0)$, and $p(0)$ are uniformly distributed within physiological bounds, and the initial rates $r_\psi(0)$, $r_\theta(0)$, and $a(0)$ are known to be zero.

The initial states $[x(0), y(0)]^\top$, $\psi(0)$, $\theta(0)$, $p(0)$, the process noise sequences $\{v_{r_\psi}(\cdot)\}$, $\{v_{r_\theta}(\cdot)\}$, $\{v_a(\cdot)\}$, and the sonar noise sequences $\{w_{S,x}(\cdot)\}$, $\{w_{S,y}(\cdot)\}$, $\{w_{S,z}(\cdot)\}$ are mutually independent. Furthermore, for each $k > 0$, given $z(k)$, $w_{U,x}(k)$ and $w_{U,y}(k)$ are conditionally independent of each other and of all initial states, process noises, and other measurement noises.

[1] For your EKF implementation, ignore the clipping for both the prior mean update and the linearization, i.e., treat (3) as $z(k) = z(k\text{–}1) + p(k\text{–}1)\sin(\theta(k\text{–}1))T_s$.

Noise Distribution

In the lecture, we often assume purely Gaussian noise distributions for our filtering problems. However, in real-world applications, this is often not the case. In this programming exercise, we provide two different sets of noise distributions. You may compare the performance of both estimation algorithms on the two different noise models. The noise models are designed such that the mean and variance are identical for both cases.

Gaussian Noise

The process noises for the whale's maneuvers are defined as $v_{r_\psi}(k) \sim \mathcal{N}\left(0, \sigma_{r_\psi}^2\right)$, $v_{r_\theta}(k) \sim \mathcal{N}\left(0, \sigma_{r_\theta}^2\right)$, and $v_a(k) \sim \mathcal{N}\left(0, \sigma_a^2\right)$.

The USBL sensor shares the same state dependent noise model in both horizontal directions. The horizontal measurement noises are modeled as zero-mean Gaussians, $w_{U,x}(k) \sim \mathcal{N}\left(0, \sigma_U^2(z(k))\right)$ and $w_{U,y}(k) \sim \mathcal{N}\left(0, \sigma_U^2(z(k))\right)$. However, the USBL sensor's accuracy is dependent on the depth of the corresponding transponder, hence the standard deviation scales linearly with the whale's depth:

$$\sigma_U(z(k)) = \sigma_{\text{base}} + \alpha z(k). \tag{16}$$

Furthermore, at or below a specific depth (z_{drop}), the USBL sensor experiences a complete dropout, and the measurements are recorded as `np.nan`.

The sonar sensor shares the same model in all directions, where $w_{S,x}(k) \sim \mathcal{N}\left(0, \sigma_S^2\right)$, $w_{S,y}(k) \sim \mathcal{N}\left(0, \sigma_S^2\right)$, and $w_{S,z}(k) \sim \mathcal{N}\left(0, \sigma_S^2\right)$.

Non-Gaussian Noise

The process noise is defined as $v_{r_\psi}(k) \sim \mathcal{U}\left(-\sqrt{3}\sigma_{r_\psi}, \sqrt{3}\sigma_{r_\psi}\right)$, $v_{r_\theta}(k) \sim \mathcal{U}\left(-\sqrt{3}\sigma_{r_\theta}, \sqrt{3}\sigma_{r_\theta}\right)$, and $v_a(k) \sim \mathcal{U}\left(-\sqrt{3}\sigma_a, \sqrt{3}\sigma_a\right)$.

As with the Gaussian case, the USBL sensor shares the same state dependent noise model in both horizontal directions. Horizontal measurement noises are modeled as uniform distributions $w_{U,x}(k) \sim \mathcal{U}\left(-\sqrt{3}\sigma_U(z(k)), \sqrt{3}\sigma_U(z(k))\right)$ and $w_{U,y}(k) \sim \mathcal{U}\left(-\sqrt{3}\sigma_U(z(k)), \sqrt{3}\sigma_U(z(k))\right)$. The sensor exhibits the same degrading performance scaling linearly with depth as shown in Equation 16, along with the same dropout depth at z_{drop}, with the measurements also being recorded as `np.nan`.

The sonar sensor shares the same model in all directions ($w_{S,x}$, $w_{S,y}$, and $w_{S,z}$). We use the placeholder subscript $(\cdot)$ to indicate that this noise model applies equally to any single axis, where $w_{\text{sonar},(\cdot)}(k)$ is an asymmetric mixture of two Gaussian distributions. For ease of notation, we will drop the time index k. Let

$$a \sim \mathcal{N}\left(\beta\sigma_S, (1-9\beta^2)\sigma_S^2\right) \text{ and } b \sim \mathcal{N}\left(-9\beta\sigma_S, (1-9\beta^2)\sigma_S^2\right)$$

be two GRVs, where $\beta = 0.3$. The PDF of $w_{S,(\cdot)}$ is given by:

$$p_{w_{S,(\cdot)}}\left(\bar{w}_{S,(\cdot)}\right) = 0.9 p_a\left(\bar{w}_{S,(\cdot)}\right) + 0.1 p_b\left(\bar{w}_{S,(\cdot)}\right). \tag{17}$$

Equivalently, we can write the PDF as:

$$\begin{aligned} p\left(w_{S,(\cdot)}\right) = \frac{1}{\sqrt{1-9\beta^2}\sigma_S\sqrt{2\pi}}\Bigg[&0.9\exp\left(-\frac{1}{2}\left(\frac{w_{S,(\cdot)} - \beta\sigma_S}{\sqrt{1-9\beta^2}\sigma_S}\right)^2\right) \\ &+0.1\exp\left(-\frac{1}{2}\left(\frac{w_{S,(\cdot)} + 9\beta\sigma_S}{\sqrt{1-9\beta^2}\sigma_S}\right)^2\right)\Bigg]. \end{aligned} \tag{18}$$

The resulting PDF is illustrated in Figure 2.

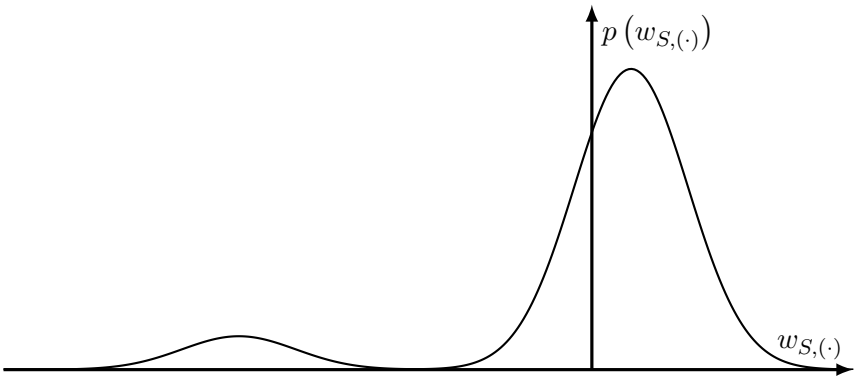

Figure 2: Probability density function of the asymmetric sonar measurement noise with $\beta = 0.3$.

Provided Files

A set of files is provided on the class website. Please use them to solve the exercise.

`estimator.py`	Python class template to be used for your implementation of estimation algorithms. **This is the only file you need to modify.**
`run.sh` / `run.bat`	Launchers to build and start the Docker container for Linux/macOS and Windows, respectively. These scripts also set up a local Python virtual environment to display the plots on the host machine.
`run.py`	Python script executed inside the Docker container. It runs the simulation of the system, runs your estimator, scores the results, and saves the output data to `results.npz`.
`simulator.py`	Python class used to simulate the motion of the whale and measurements. This class is called by `run.py`, and is obfuscated (i.e., its source code is not readable). It relies on the `pyarmor_runtime_000000` directory.
`const.py`	Constants known to the estimator, including initial state bounds, process noise coefficients, sensor noise characteristics, dropout depth, drag coefficient, discrete time step parameters, and the deterministic behavioral model coefficients driving the whale's motion.
`scoring.py`	Python script containing RMSE and timing coefficient scoring functions that are used for the evaluation.
`plot_results.py`	Python script used to load the saved simulation results (`results.npz`) and display the interactive plots on your host machine.
`plotting.py`	Python script containing helper functions for generating plots of the trajectory, states, and measurements.
`Dockerfile`	Creates a Docker image to run the simulator and estimator in an isolated environment with the exact same CPU (4 cores) and RAM (2 GB) limits as the final grading system.
`entrypoint.sh`	Initialization script used by the Docker container to run simulation and estimation scripts and gracefully exit.
`requirements.txt`	List of Python packages required to run simulation and plot results (installed automatically by `run.sh` / `run.bat`).

Task

Design an EKF for the Gaussian scenario along with any tracking algorithm of your choice for the Non-Gaussian case to estimate the 9-dimensional state vector, your EKF algorithm must also return the posterior state covariance $P_m(k)$ at each timestep. Implement your solutions for the estimator in the file `estimator.py`. Your code has to run with the Python script `run.py`. You *must* successfully handle the state-dependent USBL noise covariance, as well as the USBL sensor dropouts (represented by `np.nan`) when the whale dives below the USBL tracking threshold z_{drop}. You *must* use *exactly* the definition as given in the template `estimator.py` for the implementation of your estimator.

To ensure compatibility with the obfuscated `simulator.py` file and accurately replicate the automated grading environment, execute the code using the provided Docker container. You are only allowed to use the Python standard library and the packages pre-installed within this container.

Prerequisites:

- **Docker Desktop** — `https://www.docker.com/products/docker-desktop`
  - Windows: Make sure "Use WSL 2" is checked during install.
  - macOS/Linux: Standard install.
- **Python 3.8+** — `https://www.python.org/downloads/`

Execution Workflow:
To run the exercise, navigate to the exercise folder in your terminal and execute the launcher script:

- **Windows:** `run.bat`
- **macOS / Linux:** `bash run.sh`

Once the plots are displayed, you do not need to manually restart the script from the terminal after every change. You can keep the plot windows open, make edits to `estimator.py`, save the file, and simply click the *Rerun Simulation* button located on the 3D plot. This will automatically reload your updated code and restart the simulation and estimation pipeline.

While the style of your code is not relevant for evaluation, points will be deduced for severe violation of common sense programming techniques, which result, for example, in considerably increased computation times.

Evaluation

To evaluate your solution, we will test your assigned EKF and chosen algorithm on the given problem setup. The `Gaussian` noise will be used to evaluate the EKF, while the `Non-Gaussian` noise will be used for the algorithm you have chosen yourself. Moreover, we will make suitable modifications to the parameters in `const.py` and test the robustness of your estimator for different values of the constants. Specifically, the values of the constants R, σ_{base}, α, σ_S, and ω_{yaw} will be modified, where the modification range is given in `const.py`.

The correctness, efficiency, and robustness of your method will be key for the evaluation. Specifically, we will evaluate the following requirements:

- **Correctness:** Your EKF solution will be evaluated purely for algorithmic correctness, yielding a binary pass/fail score $b_{\text{corr}} \in \{0, 1\}$ (where 1 indicates a correct implementation and 0 indicates an incorrect one). This is measured by comparing your filter's estimated posterior mean and covariance at each timestep to that of a correctly implemented reference EKF. You will not have access to this evaluation function.

- **Non-Gaussian Estimator Accuracy:** The accuracy of your chosen Non-Gaussian algorithm is calculated as the Root Mean Squared Error (RMSE) across all 9 states. For standard linear states, the RMSE is calculated as:

$$\text{RMSE}_j = \sqrt{\frac{1}{N}\sum_{k=1}^{N}\left(\hat{x}_j(k) - x_j(k)\right)^2} \tag{19}$$

To properly evaluate the angular states, yaw (ψ) and pitch (θ), we account for angular wrapping by computing the shortest angular distance:

$$\text{RMSE}_{\text{angle}} = \sqrt{\frac{1}{N}\sum_{k=1}^{N}\text{wrap}\left(\hat{\theta}(k) - \theta(k)\right)^2} \tag{20}$$

where wrap$(\cdot)$ maps the angle difference to the valid interval $[-\pi, \pi)$.

We then normalize each of your Non-Gaussian filters' states by the class median for that specific state. The overall error coefficient is calculated as follows:

$$\varepsilon_{\text{norm}} = \frac{1}{9}\sum_{j=1}^{9}\frac{\text{RMSE}_j}{m_j} \tag{21}$$

where m_j is the *median* RMSE for state j computed across all student submissions.

- **Computational Time:** Both estimators are evaluated for efficiency using a timing coefficient defined by a sigmoid function. This scores the average step time t_{avg} against a specified maximum time threshold T (which takes on value T_{EKF} for the EKF, and T_{NGE} for the estimator receiving Non-Gaussian noise):

$$c_{\text{time}} = \frac{1}{1 + \exp\left(\lambda\frac{t_{\text{avg}} - T}{T}\right)} \tag{22}$$

where λ determines the steepness of the scoring drop-off once the maximum allowed time is reached and exceeded.

- **Final Score:** The overall evaluation score (where a higher score indicates better performance) combines the EKF's time-weighted correctness with the Non-Gaussian estimator's time-weighted accuracy ratio:

$$\text{Final Score} = (b_{\text{corr}} \cdot c_{\text{time, EKF}}) + \exp\left(-\ln(2) \cdot \varepsilon_{\text{norm}}^2\right) \cdot c_{\text{time, NGE}} \tag{23}$$

- In the case of a draw after the previous point calculation, the submission received first will be rewarded with more points.

Submission

Your submission must follow the instructions here, as grading is automated. Submissions that do not conform will not be graded.

Hand in a single zip-file, where the filename contains your name according to this template (note that there are no spaces in the filename):

`RE26_FirstnameSurname.zip`

The zip-file contains the `RE26_FirstnameSurname` folder (with the correct name as in the zip filename). This folder only contains one file:

- Your implementation of estimation algorithms in the single file `estimator.py`, which has the exact same function definition as in the provided template.

Send your zip-file in a single email:

- Send the email to `recursiveestimation@ethz.ch`.
- Use this *exact* subject: `Programming Exercise Submission - Firstname Surname`

You will receive an automatic reply confirming receipt of the email for your first submission. The teaching assistants will not look at the email before the deadline expires, and you are ultimately responsible that we correctly receive your solution in time. The submission window closes at 23:59 on the day of the deadline. Late submissions will not be considered, nor will submissions where the attachment was forgotten. You may submit the solutions multiple times but please follow the deliverable requirements strictly. Subsequent submissions will no longer receive confirmation receipts. Only the last submission will be evaluated.

No teamwork is allowed. Submissions with identical codes will not be graded.